



TECHNIQUES DE TEST

Sarra MEJRI:

- Docteur en informatique
- sarra.mejri.isim@gmail.com

Test des logiciels
certification ISTQB



PLAN

- 1. Catégories de techniques de test**
- 2. Techniques de test boîte-noire**
- 3. Techniques de test boîte-blanche**
- 4. Techniques de test basées sur l'expérience**

CATÉGORIES DE TECHNIQUES DE TEST



OBJECTIF D'UNE TECHNIQUE DE TEST

- L'objectif d'une technique de test est d'aider à identifier :
 - les conditions de test,
 - les cas de test
 - et les données de test

CHOIX DES TECHNIQUES DE TEST

- Choix des techniques de test à utiliser dépend d'un certain nombre de facteurs : type de composant ou de système, complexité du composant ou des systèmes, normes réglementaires, exigences client ou contractuelles, niveaux de risque, types de risques, objectifs du test, documentation disponible, connaissances et compétences des testeurs, outils disponibles, temps et budget, modèle de cycle de vie du développement logiciel, types de défauts ...
- Certaines techniques sont plus applicables à certaines situations et à certains niveaux de test ; d'autres sont applicables à tous les niveaux de test.
- Lors de la création de cas de test, les testeurs utilisent généralement une combinaison de techniques de test pour obtenir les meilleurs résultats de l'effort de test

CATÉGORIES DES TECHNIQUES DE TESTS

Techniques boîte-noire

se concentrent sur les entrées et sorties de l'objet de test sans référence à sa structure interne.

Techniques boîte-blanche

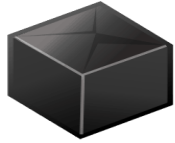
se concentrent sur la structure et le traitement à l'intérieur de l'objet de test.

Techniques basées sur l'expérience

tirent parti de l'expérience des développeurs, des testeurs et des utilisateurs pour concevoir, implémenter et exécuter des tests.

LES TECHNIQUES DE TEST

Selon le syllabus – niveau de fondation



techniques boîte-noire

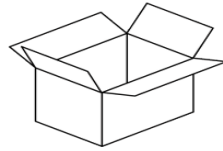
Partitions d'équivalence

Analyse des valeurs limites

Tests de table de decision

Tests de transitions d'état

Tests des cas d'utilisation



techniques boîte-blanche

Test d'instructions

Tests de décisions



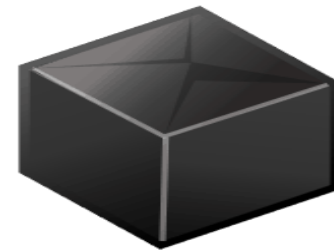
Techniques basées sur l'expérience

Estimation d'erreur

Tests exploratoires

Tests basés sur les checklists

TECHNIQUES DE TEST BOÎTE-NOIRE



TECHNIQUES DE TEST BOÎTE-NOIRE

Partitions
d'équivalence

Analyse des
valeurs limites

Tests de table de
decision

Tests de transitions
d'état

Tests des cas
d'utilisation

Partitions d'équivalence

*Principe, couverture et
exemples*

PARTITIONS D'ÉQUIVALENCE

Principe

Les partitions d'équivalence divisent les données en partitions (également connues sous le nom de classes d'équivalence) de telle sorte que tous les éléments d'une partition donnée sont supposés être traités de la même manière.

PARTITIONS D'ÉQUIVALENCE - VALIDITÉ

Valeurs valides et invalides !

- Il y a des partitions d'équivalence pour les valeurs valides et invalides.
 - Les valeurs valides sont des valeurs qui doivent être acceptées par le composant ou le système. Une partition d'équivalence contenant des valeurs valides est appelée "partition d'équivalence valide".
 - Les valeurs invalides sont des valeurs qui doivent être rejetées par le composant ou le système. Une partition d'équivalence contenant des valeurs invalides est appelée "partition d'équivalence invalide".

PARTITIONS D'ÉQUIVALENCE

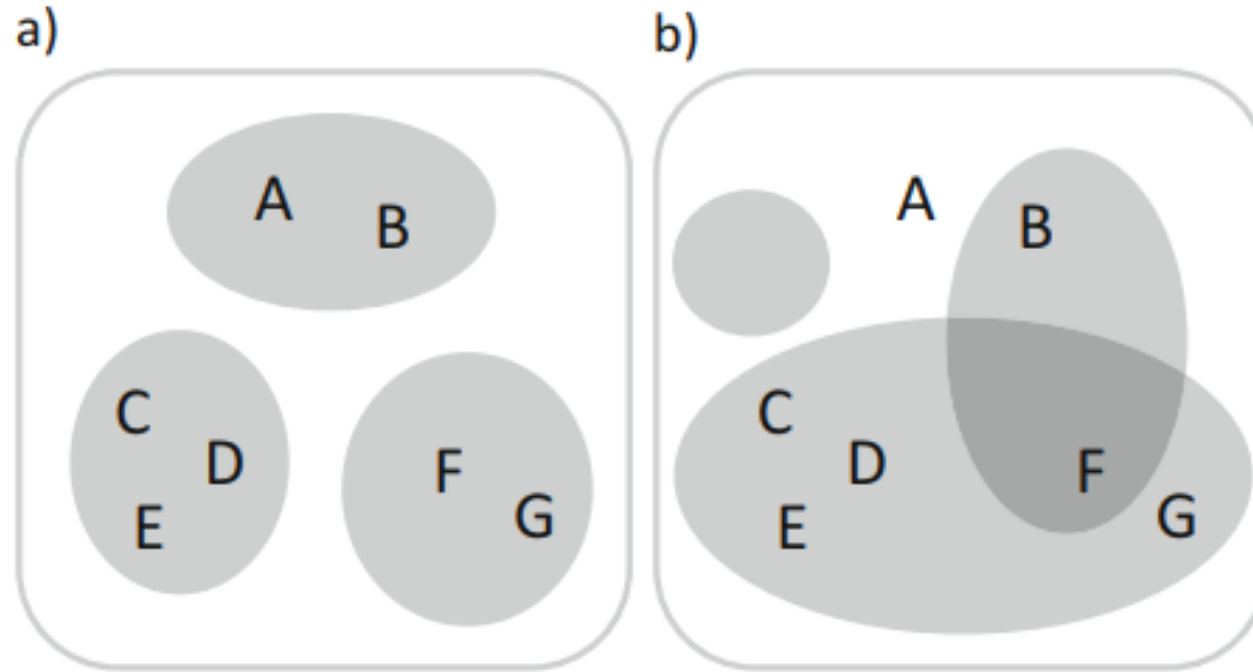
- Les partitions peuvent être identifiées pour tout élément de données lié à l'objet de test, y compris les entrées, les sorties, les valeurs internes, les valeurs liées au temps (par exemple, avant ou après un événement) et pour les paramètres d'interface (par exemple, les composants intégrés testés pendant les tests d'intégration).
- Toute partition peut être divisée en sous-partitions si nécessaire.
- **Chaque valeur doit appartenir à une et une seule partition d'équivalence.**
- Lorsque des partitions d'équivalence invalides sont utilisées dans des cas de test, elles doivent être testées individuellement, c'est-à-dire qu'elles ne doivent pas être combinées avec d'autres partitions d'équivalence invalides, afin de s'assurer que les défaillances ne sont pas masquées.
- Les défaillances peuvent être masquées lorsque plusieurs défaillances surviennent en même temps, mais qu'une seule est visible, ce qui fait que les autres défaillances ne sont pas détectées.

PRINCIPE

Ordonné ou non ordonné

- applicable à tous les niveaux de test: unitaire, intégration, système et acceptation;
- adaptée lorsque tous les éléments d'un même ensemble de valeurs sont supposés être traités de la même façon. Notons que l'ensemble peut être ordonné ou non:
 - exemple d'ensemble ordonné: une application prenant en entrée l'âge d'un individu.
 - exemple d'ensemble non ordonné: une application prenant en entrée des formes géométriques de différentes dimensions → Carré, Cercle, Triangle, Rectangle, ...

CORRECTE OU INCORRECTE



1. Il y a une partition vide.
2. L'élément A n'appartient à aucune partition.
3. L'élément F appartient à deux partitions différentes

PROCÉDURE

Comment faire les partitions

- 1. Identifier les partitions de valeurs valides et invalides en entrée**
- 2. Dériver les cas de test à partir des partitions**

COUVERTURE

- Pour obtenir une couverture de 100% avec cette technique, les cas de test doivent couvrir toutes les partitions identifiées (y compris les partitions invalides) en utilisant au moins une valeur de chaque partition.
- La couverture est mesurée comme étant le nombre de partitions d'équivalence testées par au moins une valeur, divisé par le nombre total de partitions d'équivalence identifiées, généralement exprimé en pourcentage. Les partitions d'équivalence sont applicables à tous les niveaux de test.

APPLICATION 1

énoncé

Considérez le domaine $\{1, 2, 3, 4\}$.

1. Notez tous les partitionnements possibles de ce domaine.
2. Supposons qu'un système accepte une valeur de cet ensemble et que l'action suivante dépend du fait que la valeur d'entrée est paire ou impaire. Quel partitionnement d'équivalence est le plus raisonnable pour ce problème ?

APPLICATION 1

corrigé

- | | | | |
|------------------------------|------------------------------|----------------------------------|------------------------------|
| 1. $\{1, 2, 3, 4\}^1$ | 2. $\{1, 2, 3\}, \{4\}$ | 3. $\{1, 2, 4\}, \{3\}$ | 4. $\{1, 3, 4\}, \{2\}$ |
| 5. $\{2, 3, 4\}, \{1\}$ | 6. $\{1, 2\}, \{3, 4\}$ | 7. $\{1, 3\}, \{2, 4\}$ | 8. $\{1, 4\}, \{2, 3\}$ |
| 9. $\{1, 2\}, \{3\}, \{4\}$ | 10. $\{1, 3\}, \{2\}, \{4\}$ | 11. $\{1, 4\}, \{2\}, \{3\}$ | 12. $\{2, 3\}, \{1\}, \{4\}$ |
| 13. $\{2, 4\}, \{1\}, \{3\}$ | 14. $\{3, 4\}, \{1\}, \{2\}$ | 15. $\{1\}, \{2\}, \{3\}, \{4\}$ | |

APPLICATION 2

énoncé

- Un système classe une voiture en fonction de son âge donné comme valeur d'entrée entière. Le système utilise les règles métier selon l'algorithme suivant :

```
INPUT CarAge
IF CarAge is incorrect
THEN Return 'error' and terminate
IF CarAge < 6
THEN Classify car as 'new'
ELSE IF CarAge <= 12
THEN Classify car as 'used'
ELSE
Classify car as 'old'
```

- Identifier les classes d'équivalence valides et invalides.

APPLICATION 2

corrigé

#	Partition	Valide/invalid	Exemple de cas de test	output attendu
1	{... , -3, -2, -1}	invalid	-5	Erreur
2	{0, 1, 2, 3, 4, 5}	valide	4	nouveau
3	{6, 7, 8, 9, 10, 11, 12}	valide	8	D'occasion
4	{13, 14, 15, ...}	valide	13	vielle

APPLICATION 3

énoncé

- Au cours du processus d'enregistrement du compte, un utilisateur choisit son statut dans une liste déroulante. Les options possibles sont: enfant, jeune, adulte. Le comportement du système dépend de l'option qui sera sélectionnée. Supposons que vous analysiez cette variable par rapport au partitionnement par équivalence.
- Qu'est-ce que le domaine et à quoi devrait ressembler son partitionnement ?

APPLICATION 3

corrigé

- Le domaine d'intérêt est le domaine de la variable "statut de l'utilisateur", car le comportement du système en dépend.
- Le domaine a trois éléments: {enfant, jeune, adulte}, et chaque valeur forme une classe d'équivalence distincte.
- La partition: {enfant}, {jeune}, {adulte}
- Par conséquent, nous devrions fournir trois cas de test : un pour le statut=enfant, un pour le statut= jeune et un pour le statut=adulte.

Analyse des valeurs limites

Principe et exemples

ANALYSE DES VALEURS LIMITES (AVL)

principe

- La technique AVL est, comme la technique PE, orientée domaines.
- L'analyse des valeurs limites est **une extension des partitions d'équivalence**.
- La différence entre AVL et PE, pour chaque classe d'équivalence:
 - Dans PE, on choisit **arbitrairement** un élément (**cas de test**);
 - Dans AVL, on choisit **les valeurs limites**.
- Les valeurs minimale et maximale (ou première et dernière valeurs) d'une partition sont **ses valeurs limites**.
- L'analyse des valeurs limites peut être appliquée à tous les niveaux de test.
- Cette technique est généralement utilisée pour tester les exigences qui nécessitent une série de valeurs numériques (y compris les dates et les heures).

ANALYSE DES VALEURS LIMITES (AVL)

principe

- AVL ne peut être utilisée que lorsque:
 - **la partition est ordonnée (ie. On peut les comparer en utilisant les opérateurs $<, >, <=, >=, =$)**
 - **composée de données numériques ou séquentielles.**
- Exemple :
 - On ne peut pas appliquer la AVL au domaine {Linux, Windows, iOS} représentant les systèmes d'exploitation, car on ne peut pas les comparer.
 - **Chaque classe doit être « compacte » :**
 - Formellement, cela signifie que si a, b appartiennent à une certaine classe, alors nécessairement tous les éléments x , tels que $a < x < b$, doivent aussi appartenir à cette classe.
 - De manière informelle, on peut dire que les classes ne peuvent pas avoir de « trous ».

ANALYSE DES VALEURS LIMITES (AVL)

Valide et invalide

- Comme la technique AVL est basée sur la technique PE,
- Nous avons également des classes d'équivalence valides et invalides :
 - Les valeurs limites des classes valides sont appelées **valeurs limites valides**.
 - Les valeurs limites des classes invalides sont appelées **valeurs limites invalides**.

ANALYSE DES VALEURS LIMITES

Méthode

- identifier les PEs / les CEs appropriées
- identifier les valeurs limites
- sélectionner les valeurs de test qui leur correspondent.

ANALYSE DES VALEURS LIMITES

2 valeurs et 3 valeurs

- Il existe deux variantes de la méthode AVL: 2 valeurs et 3 valeurs.
- Dans une version à 2 valeurs, pour chaque classe d'équivalence identifiée, les valeurs de test sont :
 - Les valeurs limites de cette classe (c'est-à-dire les éléments minimal et maximal)
 - L'élément juste en dessous de l'élément minimal
 - L'élément juste au-dessus du maximum
- Notez que les deux derniers éléments sont également les valeurs limites pour les classes d'équivalence adjacentes.
- Pour une couverture de 100 % dans la version à AVL à 2 valeurs, il nous suffit d'identifier toutes les valeurs limites pour toutes les classes identifiées et les prendre comme entrées de test.

ANALYSE DES VALEURS LIMITES

2 valeurs et 3 valeurs

Dans une version à 3 valeurs, pour chaque classe d'équivalence identifiée, les valeurs de test sont:

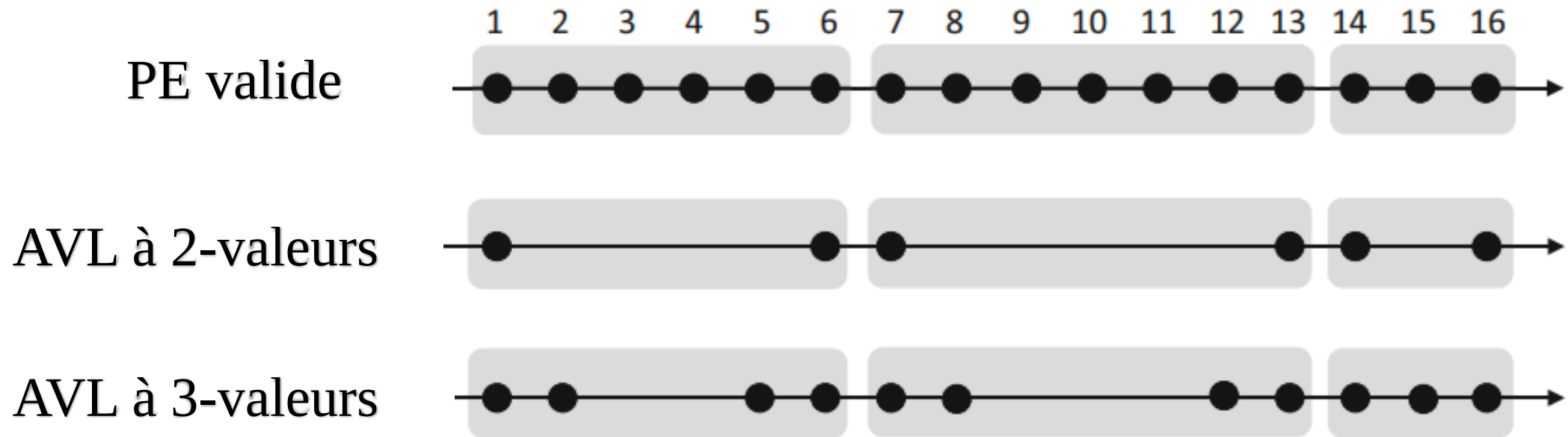
- Les valeurs limites de cette classe (c'est-à-dire les éléments minimal et maximal)
- Les éléments juste en dessous et juste au dessus du minimal
- Les éléments juste en dessous et juste au dessus du maximum

COUVERTURE

- La couverture des limites d'une partition est mesurée comme le nombre de valeurs limites testées, divisé par le nombre total de valeurs limites identifiées, généralement exprimé en pourcentage.

EXEMPLE

On considère un domaine $\{1, 2, \dots, 15, 16\}$, qui est divisé en trois classes d'équivalence valides: $\{1, \dots, 6\}$, $\{7, \dots, 13\}$, et $\{14, \dots, 16\}$.



EXEMPLE

#	PE	Valeurs limites	2-valeurs	3-valeurs
1	{1,...,6}	1,6	0 (s'il existe),1,6,7	0 (s'il existe), 1,2, 5,6,7
2	{7, ..., 13}	7,13	6,7,13,14	6,7,8,12,13,14
3	{14, . . . , 16}	14,16	13,14,16,17 (s'il existe)	13,14,15,16,17

Cas de test (AVL – 2 valeurs) :
0 (si possible) ; 1 ; 6; 7; 13,14, 16,17 (si possible)

Cas de test (AVL – 3 valeurs) :
0 (si possible), 1, 2, 5, 6, 7, 8, 12, 13, 14, 15, 16, 17 (si possible).

- 0 et 17 sont appelés les valeurs limites invalides.

APPLICATION

énoncé

Vous testez le système de notation automatique des écoles. Il prend le nombre de points collectés par un élève et renvoie automatiquement le score final de l'élève selon les règles suivantes :

Un élève peut obtenir un score compris entre 0 et 100. Un score est un nombre entier.

Le calcul du score selon les notes est le suivant :

- Plus de 90 points : A
- Plus de 75 points : B
- Plus de 60 points : C
- Plus de 49 points : D
- Sinon : F

En supposant que l'entrée peut être comprise entre 0 et 100, identifiez les classes d'équivalence et les valeurs limites pour les méthodes des valeurs limites (AVL à 2-valeurs et à 3-valeurs).

APPLICATION

corrigé

- Les classes d'équivalence doivent représenter les notes, nous avons donc les classes suivantes :
 - $A = \{91, 92, \dots, 99, 100\}$ —représentant les valeurs pour A
 - $B = \{76, 77, \dots, 89, 90\}$ —représentant les valeurs de B
 - $C = \{61, 62, \dots, 74, 75\}$ —représentant les valeurs de C
 - $D = \{50, 51, \dots, 59, 60\}$ —représentant les valeurs de D
 - $F = \{0, 1, \dots, 48, 49\}$ —représentant les valeurs de F
- Limites et valeurs limites pour la méthode des valeurs à 2 et 3 points

Classe	LIMITES	2-point VALEURS LIMITES	3-point VALEURS LIMITES
F	0, 49	0, 49, 50	0, 1, 48, 49, 50
D	50, 60	49, 50, 60, 61	49, 50, 51, 59, 60, 61
C	61, 75	60, 61, 75, 76	60, 61, 62, 74, 75, 76
B	76, 90	75, 76, 90, 91	75, 76, 77, 89, 90, 91
A	91, 100	90, 91, 100	90, 91, 92, 99, 100

- pour la méthode de la valeur à 2 points : 0, 49, 50, 60, 61, 75, 76, 90, 91, 100
- pour la méthode de la valeur à 3 points : 0, 1, 48, 49, 50, 51, 59, 60, 61, 62, 74, 75, 76, 77, 89, 90, 91, 92, 99, 100

Notez que nous n'avons pas les valeurs -1 et 101, car, selon la spécification, elles sont irréalisables

Tests de tables de décision

Principe, Couverture et exemples

TECHNIQUES DE TESTS COMBINATOIRES

- Les techniques de test combinatoire sont utiles pour tester la mise en œuvre des exigences du système qui spécifient comment différentes combinaisons de conditions donnent des résultats différents.
- L'une de ces techniques de test est le test de tables de décision.

PRINCIPE

- Les tables de décision sont un bon moyen pour répertorier les règles métier complexes qu'un système doit mettre en œuvre.
- Lors de la création de tables de décision, le testeur identifie les **conditions (souvent des entrées)** et les **actions résultantes (souvent des sorties)** du système. Celles-ci forment **les lignes du tableau**, généralement avec **les conditions en haut** et **les actions en bas**.
- Chaque **colonne** correspond à une **règle de décision** qui définit une combinaison unique de conditions qui aboutit à l'exécution des actions associées à cette règle.
- Les valeurs des conditions et des actions sont généralement affichées sous forme de valeurs booléennes (vraies ou fausses) ou de valeurs discrètes (par exemple, rouge, vert, bleu), mais peuvent également être des nombres ou des plages de nombres.
- Ces différents types de conditions et d'actions peuvent être regroupés dans la même table.

NOTATION COURANTE = TABLES DE DÉCISION

Pour les conditions :

- V signifie que la condition est vraie (peut aussi être représentée par O (Oui) ou 1)
- F signifie que la condition est fausse (peut aussi être affiché comme N (Non) ou 0)
- — signifie que la valeur de la condition n'a pas d'importance (peut également être affichée en tant que N/A)

Pour les actions :

- X signifie que l'action doit avoir lieu (peut aussi être représenté par V, O ou 1)
- Vide signifie que l'action ne doit pas se produire (peut également être affiché sous la forme - ou F, N ou 0 .

CONSTRUCTION DES TABLES DE DÉCISION

procédure

1. Identifier toutes les conditions possibles.
2. Identifier toutes les actions correspondantes qui peuvent se produire dans le système.
3. Générer toutes les combinaisons possibles de conditions: chaque combinaison unique forme une colonne distincte dans la table de décision.
4. Pour chaque combinaison de conditions, identifier les actions qui doivent être satisfaites et celles qui ne le sont pas ; remplir les champs correspondants dans une colonne donnée sous la combinaison de conditions correspondante.
5. Pour chaque colonne, concevoir un cas de test dans lequel une combinaison donnée de conditions est vérifiée ; le test doit vérifier si les actions correspondantes tiennent

COUVERTURE (1)

- Une table de décision complète comporte suffisamment de colonnes pour couvrir toutes les combinaisons de conditions.
- La table peut être réduite en supprimant les colonnes contenant des combinaisons impossibles de conditions, les colonnes contenant des combinaisons de conditions possibles mais irréalisables et les colonnes qui testent des combinaisons de conditions qui n'affectent pas le résultat.

COUVERTURE (2)

- La couverture minimale courante pour le test de tables de décision est d'avoir au moins un cas type par règle de décision dans la table. Il s'agit généralement de couvrir toutes les combinaisons de conditions.
- La couverture est mesurée comme le nombre de règles de décision testées par au moins un cas de test, divisé par le nombre total de règles de décision, généralement exprimé en pourcentage.
- La force de la technique de test utilisant les tables de décision est qu'elle permet d'identifier toutes les combinaisons importantes de conditions, dont certaines pourraient sinon être négligées. Cette technique permet également de trouver des lacunes dans les exigences.
- Elle peut être appliquée à toutes les situations dans lesquelles le comportement du logiciel dépend d'une combinaison de conditions, à n'importe quel niveau de test

EXEMPLE (1)

SPÉCIFICATION DES EXIGENCES d'un système qui vérifie si un candidat remplit toutes les conditions pour recevoir le permis

- L'opérateur saisit les informations suivantes dans le système :
 - Le résultat de l'examen théorique (une valeur entière comprise entre 0 et 100 points)
 - Le résultat de l'examen blanc (une valeur entière supérieure ou égale à zéro, représentant le nombre d'erreurs commises par le candidat lors de l'examen blanc)
- Un candidat reçoit le permis de conduire s'il a obtenu au moins 85 points (sur 100) à l'examen théorique et s'il a commis au plus 2 erreurs lors de l'examen pratique. En cas d'échec sur l'un de ces critères, le candidat est éligible pour repasser l'examen. En cas d'échec dans les deux parties, le candidat est tenu de reprendre les cours de conduite.

EXEMPLE (2)

Application de la procédure

1) **Identifiez toutes les conditions possibles.**

Les conditions possibles sont évidemment:

- Le résultat de l'examen théorique (nombre de points)
- Le résultat de l'examen blanc (nombre d'erreurs)

Les décisions dépendront des résultats de ces examens. A partir du cahier des charges, nous savons que l'examen théorique est réussi lorsqu'un candidat reçoit au moins 85 points. L'examen blanc est réussi lorsque le nombre d'erreurs est inférieur ou égal à deux.

2) **Identifiez toutes les actions correspondantes qui peuvent se produire dans le système.**

Le système peut prendre les décisions suivantes:

- Un candidat doit-il recevoir le permis de conduire ? (valeurs possibles : OUI ou NON)
- Un candidat doit-il repasser l'examen théorique ? (valeurs possibles : OUI ou NON)
- un candidat doit-il repasser l'examen blanc ? (valeurs possibles : OUI ou NON)
- Un candidat doit-il reprendre les cours de conduite ? (valeurs possibles : OUI ou NON)

3) Générer toutes les combinaisons possibles de conditions ; chaque combinaison unique forme une colonne distincte dans la table de décision.

Chaque condition a deux valeurs possibles, nous avons donc $2 * 2 = 4$ combinaisons possibles de conditions (et donc quatre colonnes dans la table de décision) :

- ✓ Points ≥ 85 et erreurs < 2 .
- ✓ Points ≥ 85 et erreurs > 2 .
- ✓ Points < 85 et erreurs ≤ 2 .
- ✓ Points < 85 et erreurs > 2 .

EXEMPLE (3)

CONDITIONS	REGLES	1	2	3	4
Points ≥ 85 ?		V	V	F	F
Erreurs ≤ 2 ?		V	F	V	F
ACTIONS					
Recevoir le permis de conduire ?					
Repasser l'examen théorique?					
Repasser l'examen pratique?					
Reprendre cours de conduite ?					

EXEMPLE (4)

- 4) Pour chaque combinaison de conditions, identifier les actions qui doivent être satisfaites et celles qui ne le sont pas ; remplir les champs correspondants dans une colonne donnée sous la combinaison de conditions correspondante.

on identifie le comportement attendu du système :

Exemple : Colonne 1 : points ≥ 85 , erreurs ≤ 2 .

Cela signifie que la candidate a réussi les deux examens, elle doit donc obtenir son permis de conduire. Il n'est pas nécessaire de repasser les examens ou les cours de conduite. Ainsi, les quatre actions correspondantes dans la colonne 1 doivent être respectivement yes, no, no, no.

EXEMPLE (5)

CONDITIONS	REGLES	1	2	3	4
Points ≥ 85 ?		V	V	F	F
Erreurs ≤ 2 ?		V	F	V	F
ACTIONS					
Recevoir le permis de conduire ?		V	F	F	F
Repasser l'examen théorique?		F	F	V	V
Repasser l'examen pratique?		F	V	F	V
Reprendre cours de conduite ?		F	F	F	V

EXEMPLE (6)

5. Pour chaque colonne, concevoir un cas de test dans lequel une combinaison donnée de conditions est vérifiée ;
le test doit vérifier si les actions correspondantes tiennent

Pour chaque colonne, nous devons concevoir un cas de test. Un cas de test doit avoir les valeurs d'entrée concrètes. Dans notre cas, ce sera le nombre de points et le nombre d'erreurs. Les cas de test correspondants pour nos quatre colonnes dans la table de décision pourraient ressembler à ceci.

Cas de test	Input		Résultat attendu (output)
	Points	Erreurs	
1	85	0	Accorder la licence, n'oblige pas le candidat à repasser un examen ou des leçons
2	95	4	Exige que le candidat reprenne uniquement l'examen blanc
3	84	1	Requires the candidate only to retake the theory exam
4	11	3	Requires the candidate to retake both exams and the driving lessons

Tests de transitions d'états

Principe, couverture et exemples

DIAGRAMME D'ÉTAT-TRANSITION (1)

Objectif: Décrire le comportement dynamique d'une entité (logiciel, composant, objet...)

Comportement décrit par états + transitions entre les états:

- **État**: abstraction d'un moment de la vie d'une entité pendant lequel elle satisfait un ensemble de conditions
- **Transition**: changement d'état

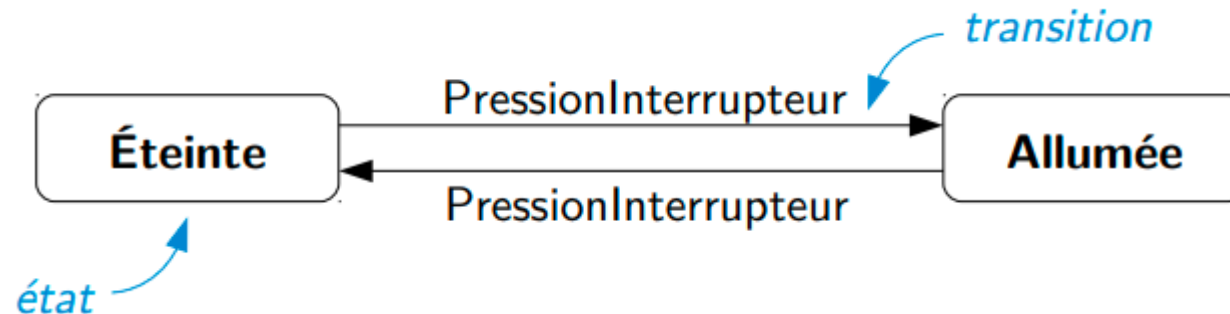


DIAGRAMME D'ÉTAT-TRANSITIONS (2)

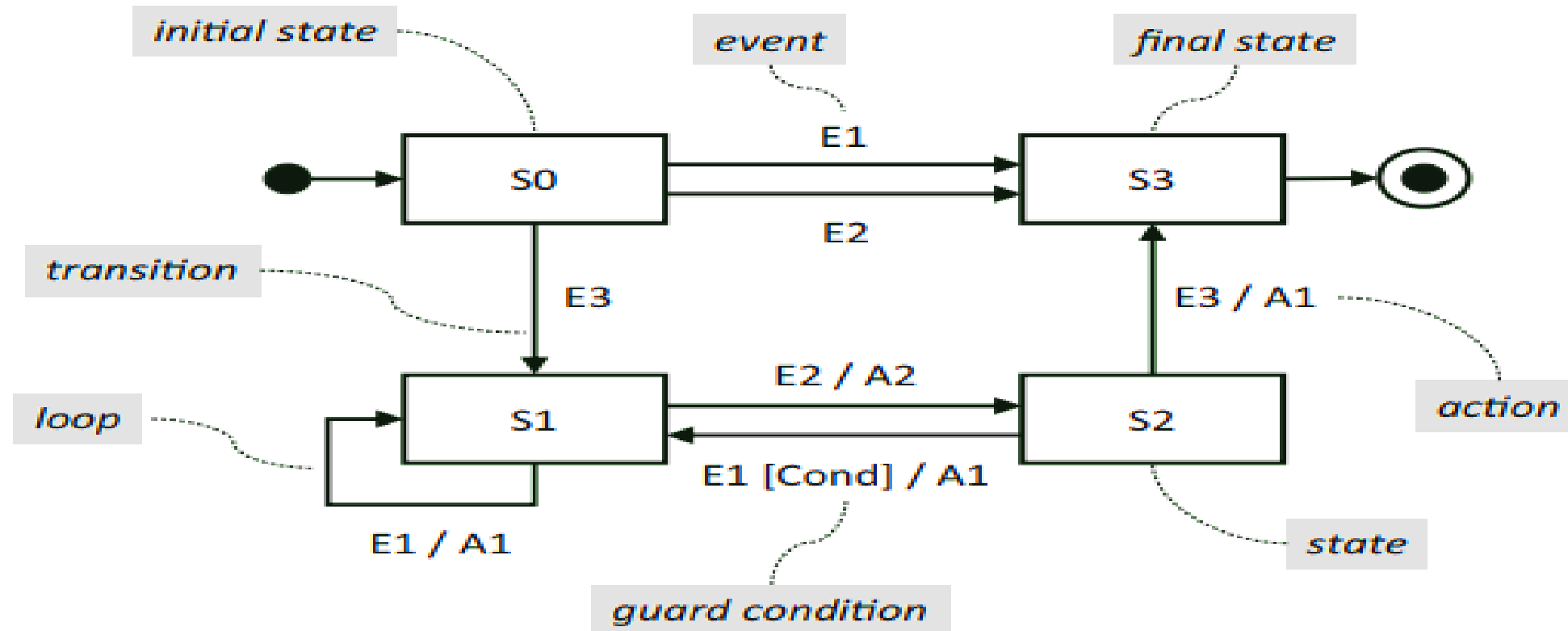
- **Un diagramme de transitions d'état** montre **les états** possibles du logiciel, ainsi que la façon dont le logiciel entre, sort et évolue entre ces états.
- **Une transition** est déclenchée par un **événement**. L'événement entraîne une transition.
 - ✓ Par exemple, l'entrée d'une valeur par l'utilisateur dans un champ.
- Si le même événement peut entraîner deux transitions différentes ou plus à partir d'un même état, cet événement peut être qualifié par **une condition de garde**. Le changement d'état peut entraîner **une action** du logiciel.
 - ✓ Par exemple, l'affichage du résultat d'un calcul ou d'un message d'erreur.

DIAGRAMME D'ÉTAT-TRANSITIONS (3)

Le diagramme de transition d'états se compose de:

- **États**: ils représentent les conditions possibles dans lesquelles le système peut se trouver.
- **Transitions**: elles représentent les changements de machine possibles d'un état à un autre.
- **Événements**: ils représentent les choses externes qui peuvent se produire et déclencher le changement d'état.
- **Actions**: elles représentent les activités possibles effectuées par la machine lors du changement d'état.
- **Conditions de garde**: elles représentent les conditions supplémentaires pour les transitions qui doivent être remplies pour déclencher la transition ; en particulier, elles sont utilisées s'il y a plus d'une transition possible du même état sous le même événement.

DIAGRAMME D'ÉTAT-TRANSITIONS (3)

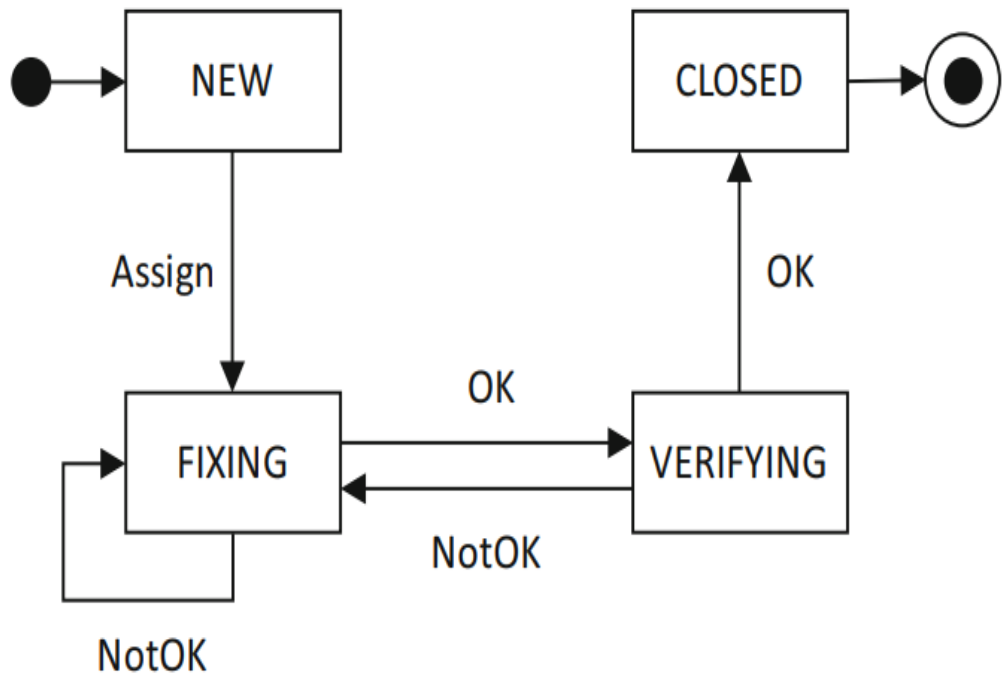


Les états, les transitions et les événements sont les composants typiques d'un diagramme d'état. Les actions et les conditions de garde sont facultatives.

TABLEAU DE TRANSITION D'ÉTAT

- Un tableau de transition d'état montre toutes les transitions valides et les transitions potentiellement invalides entre les états, ainsi que les événements, les conditions de garde et les actions résultantes pour les transitions valides.
- Les diagrammes de transition d'états ne montrent normalement que les transitions valides et excluent les transitions invalides.

EXEMPLE DE DIAGRAMME D'ÉTAT-TRANSITION ET DE TABLEAU DE TRANSITION D'ÉTAT



State \ Event	Assign	OK	NotOK
NEW	FIXING		
FIXING		VERIFYING	FIXING
VERIFYING		CLOSED	FIXING
CLOSED			

State	Event	NextState
NEW	Assign	FIXING
NEW	OK	(undef)
NEW	NotOK	(undef)
FIXING	Assign	(undef)
FIXING	OK	VERIFYING
FIXING	NotOK	FIXING
VERIFYING	Assign	(undef)
VERIFYING	OK	CLOSED
VERIFYING	NotOK	FIXING
CLOSED	Assign	(undef)
CLOSED	OK	(undef)
CLOSED	NotOK	(undef)

TEST DE TRANSITIONS D'ÉTATS

- Un tableau de transition d'état montre toutes les transitions valides et les transitions potentiellement invalides entre les états, ainsi que les événements, les conditions de garde et les actions résultantes pour les transitions valides.
- Les diagrammes de transition d'états ne montrent normalement que les transitions valides et excluent les transitions invalides.
- Les tests peuvent être conçus pour couvrir une séquence de plusieurs états, pour exercer tous les états, pour exercer toutes les transitions, pour exercer des séquences spécifiques de transitions ou pour tester des transitions invalides.
- Le test de transitions d'état est utilisé pour les applications basées sur des menus et est aussi couramment utilisé dans l'industrie du logiciel embarqué. La technique convient également à la modélisation d'un scénario métier ayant des états spécifiques ou pour tester la navigation à l'écran. Le concept d'état est abstrait, il peut représenter quelques lignes de code ou un processus métier entier.

COUVERTURE

- La couverture est généralement mesurée comme étant le nombre d'états ou de transitions identifiés testés, divisé par le nombre total d'états ou de transitions identifiés dans l'objet de test, normalement exprimé en pourcentage.
- Les critères de couverture classiques pour les diagrammes de transition d'état sont les suivants:
 - Couverture de l'État (le critère de couverture le plus faible) :
 - exige que chaque état soit visité au moins une fois dans au moins un test;
 - Les conditions de test sont tous les États.
 - Couverture de transition (appelée également la couverture 0-switch) :
 - exige que chaque transition valide doit être exécutée au moins une fois dans au moins un test
 - Les conditions de test sont toutes les transitions.
 - Couverture de transition invalide
 - exige que nous devons essayer d'exécuter chaque transition non valide dans notre suite de test
 - Les conditions de test sont toutes les transitions non valides.

COUVERTURE

- Il existe aussi un type spécial de couverture (de transitions), appelé **couverture n-switch**.
- Un n-switch est une séquence d'états telle qu'il y a exactement **n états entre le premier et le dernier dans cette séquence** (en excluant le premier et le dernier).
- Un **0-switch** (ou couverture de transition) est une séquence d'un état à un autre sans états (zéro) entre eux. Il s'agit exactement d'une seule transition entre les deux états.

COUVERTURE N-SWITCH

- Une **n-switch** est une **séquence d'états** dans laquelle **il y a exactement n états entre le premier et le dernier**.
- Définition: Si on note les états comme suit : $E_0 \rightarrow E_1 \rightarrow E_2 \rightarrow \dots \rightarrow E_n \rightarrow E_{n+1}$
- Alors la séquence est une $E_0 \rightarrow E_{n+1}$ **n-switch**, car elle contient exactement **n états intermédiaires** (E_1 à E_n).
- Exemple: Supposons qu'on ait un automate avec ces transitions : $A \rightarrow B \rightarrow C \rightarrow D$
- La séquence $A \rightarrow C$ est une **1-switch** : Il y a **1 état** entre A et C : B .
- La séquence $A \rightarrow D$ est une **2-switch** : Il y a **2 états** entre A et D : B et C .

0-SWITCH (OU COUVERTURE DE TRANSITION)

- Un **0-switch** signifie qu'il n'y a aucun état entre le premier et le dernier état de la séquence. Autrement dit, c'est **une seule transition directe entre deux états**.
- **Exemple** : Toujours avec l'automate: $A \rightarrow B \rightarrow C \rightarrow D$
- $A \rightarrow B$ est une **0-switch** :
 - Il n'y a **aucun état entre A et B**, c'est **une transition directe**.
- $B \rightarrow C$ est aussi une **0-switch**.

COUVERTURE

Type de switch	Description	Exemple (avec $A \rightarrow B \rightarrow C \rightarrow D$)
0-switch	Transition directe sans état entre	$A \rightarrow B$
1-switch	Un seul état intermédiaire	$A \rightarrow C$ (passe par B)
2-switch	Deux états intermédiaires	$A \rightarrow D$ (passe par B et C)

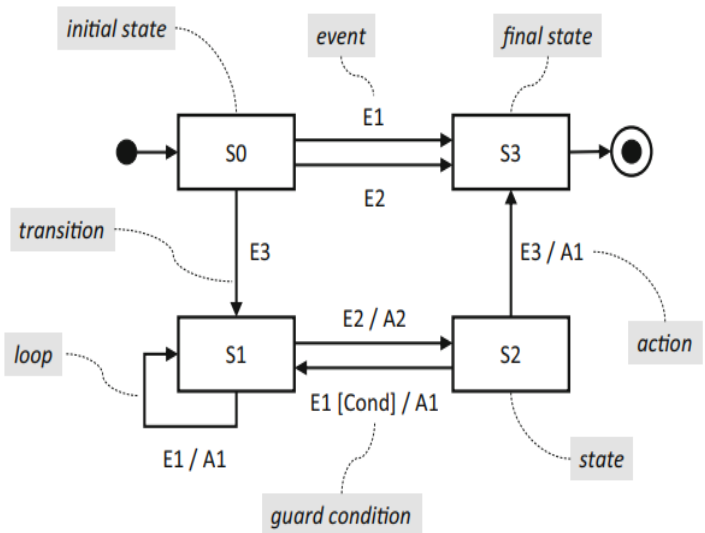
EXEMPLE

Couverture 0-switch

✓ Afin d'atteindre la couverture, nous avons d'abord besoin d'identifier toutes les transitions valides (c-à-d tous les 0-switch dans le diagramme).

- (1) S0 → (E1) → S3
- (2) S0 → (E2) → S3
- (3) S0 → (E3) → S1
- (4) S1 → (E1) → S1
- (5) S1 → (E2) → S2
- (6) S2 → (E1) [Cond] → S1
- (7) S2 → (E3) → S3

✓ Nous devons couvrir toutes ces sept transitions.

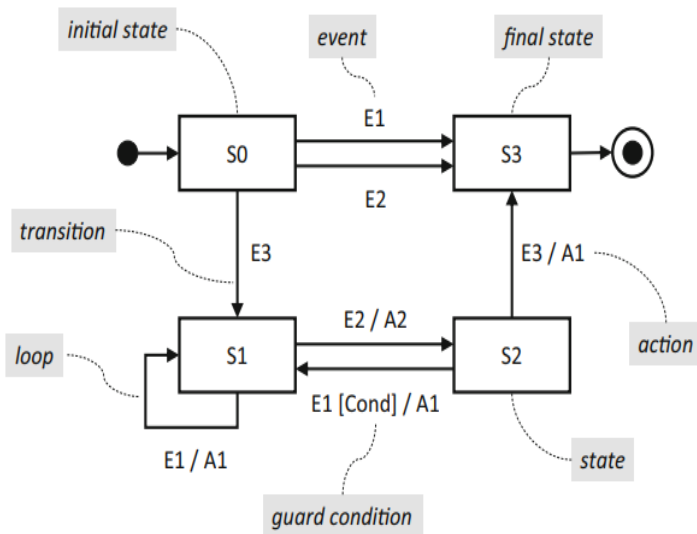


#	Cas de Test	0-switch transitions couvertes
1	S0 → (E1) → S3	(1)
2	S0 → (E2) → S3	(2)
3	S0 → (E3) → S1 → (E1) → S1 → (E2) → S2 → (E1) [Cond] → S1 → (E2) → S2 → (E3) → S3	(3), (4), (5), (6), (7)

EXEMPLE

Couverture de transitions invalides

- Dans le cas de ce critère, un test ne devrait couvrir qu'une seule transition invalide.



- (1) $S1 \rightarrow (E3) \rightarrow ?$
- (2) $S2 \rightarrow (E2) \rightarrow ?$
- (3) $S3 \rightarrow (E1) \rightarrow ?$
- (4) $S3 \rightarrow (E2) \rightarrow ?$
- (5) $S3 \rightarrow (E3) \rightarrow ?$

- Par conséquent, nous avons besoin de **cinq cas de test**.
- Dans chacun d'eux, nous essayons d'invoquer une séquence spécifique et non valide d'états.
- Par exemple, dans le cas de (2), nous devons d'abord arriver à S2, puis essayer d'invoquer l'événement E2.
 - $S0 \rightarrow (E3) \rightarrow S1 \rightarrow (E2) \rightarrow S2 \rightarrow (E2) \rightarrow ?$

Tests de cas d'utilisation

Principe, couverture et exemples

LES CAS D'UTILISATION (1)

- **Un cas d'utilisation** (« use case ») **représente un ensemble de séquences d'actions qui sont réalisées par le système et qui produisent un résultat observable intéressant pour un acteur particulier.**
- Les cas d'utilisation sont une façon spécifique de concevoir les interactions avec le logiciel pour représenter des exigences.
- Les cas d'utilisation sont associés à des **acteurs** (utilisateurs humains, matériel externe ou autres composants ou systèmes) et à des sujets (le composant ou système auquel le cas d'utilisation est appliqué).
- Chaque cas d'utilisation spécifie un comportement qu'un sujet peut accomplir en collaboration avec un ou plusieurs acteurs
- Un cas d'utilisation peut être décrit par des interactions et des activités, **ainsi que par des préconditions, des post conditions et du langage naturel.**

LES CAS D'UTILISATION (2)

- Les interactions entre les acteurs et le sujet peuvent entraîner des changements dans l'état du sujet. Les interactions peuvent être représentées graphiquement par des flux de travail, des diagrammes d'activités ou des modèles de processus métier.
- Un cas d'utilisation peut inclure des variations possibles de son comportement de base.

LES CAS D'UTILISATION (3)

corps

Le cas d'utilisation devrait se composer de

- Préconditions (facultatif)
- Post-conditions (résultats observables) [garanties de réussite]
- Un seul chemin principal (parfois appelé chemin heureux) [scénario de réussite principal]
- Chemins alternatifs (0 ou plus) [liste de variation technologique et de données]
- Chemins d'exception (0 ou plus) [extensions - erreur]

LES CAS D'UTILISATION (4)

- Il ne faut pas utiliser les branchements conditionnels dans le scénario : le chemin principal doit être une seule séquence d'étapes et ne doit contenir aucun branchement conditionnel.
- L'utilisation du diagramme de cas d'utilisation UML n'est pas suffisante pour définir correctement les cas d'utilisation. Le diagramme de cas d'utilisation UML n'est pas une notation pour capturer tous les scénarios possibles. Il présente la décomposition des cas d'utilisation, pas leur contenu.
- Lors de la dérivation des cas de test à partir du cas d'utilisation, il est recommandé de combiner la technique basée sur les cas d'utilisation avec d'autres techniques de boîte noire ;
 - par exemple, lorsque nous passons par une étape du scénario principal : « un utilisateur saisit le nombre de produits »; nous pouvons appliquer aussi AVL dans le cas de test,
 - nous pouvons exiger que l'utilisateur saisir la valeur limite (par exemple, 1 ou le maximum valeur autorisée);
 - les alternatives/exceptions doivent également être invoquées avec les valeurs limites (par exemple, 0 produits ou maximum autorisé)

EXCEPTIONS DANS LES CAS D'UTILISATION

Cas d'utilisation...

....

Etape 1 ...

Etape 2 ... (E1)

Etape 3 ...

Etape 4 ... (E1, E2)

Etape 5 ... (E3)

...

Exceptions:

E1: ...

E2: ...

E3: ...

Cas d'utilisation...

...

Etape 1 ...

Etape 2 ...

Etape 3 ...

Etape 4 ...

Etape 5 ...

...

Exceptions:

2a: ...

4a: ...

4b: ...

5a: ..

LES SCÉNARIOS

Un scénario représente une succession particulière d'enchaînements, s'exécutant du début à la fin du cas d'utilisation, un enchaînement étant l'unité de description de séquences d'actions.

Un cas d'utilisation contient en général **un scénario nominal** et plusieurs **scénarios alternatifs** (qui se terminent de façon normale) **ou d'erreur /exceptionnels** (qui se terminent en échec).

LES SCÉNARIOS

Alternatif VS Exceptionnel

- Les chemins alternatifs permettent à l'utilisateur d'atteindre son objectif en empruntant un chemin alternatif, qui est généralement plus long que le scénario principal.
 - Par exemple, dans un cas d'utilisation pour retirer de l'argent, un utilisateur saisit incorrectement le code PIN et le flux de cas d'utilisation peut exiger que l'utilisateur entre à nouveau le code PIN. Lorsque l'utilisateur sélectionne correctement le code PIN la deuxième fois et retire l'argent, elle atteint son objectif, mais avec un chemin alternatif (plus long).
- Les exceptions sont des situations imprévues qui conduisent généralement à la fin anormale du cas d'utilisation. Par exemple :
 - Un utilisateur annule son opération
 - Un système perd la connexion avec le serveur
 - Il n'y a pas suffisamment d'argent dans la machine (dans le cas de l'exemple de retrait d'argent au distributeur automatique)

EXEMPLE DE CAS D'UTILISATION

Use case #001: Enregistrer de l'arrivée d'une boîte.

Acteur principal: AA

Scope: Enregistrement de l'arrivée d'une boîte

Scénario principal :

1. AA reçoit et ouvre une boîte (ID de la boîte, sacs avec des ID de sacs) de la société de transport TC.
2. AA valide l'ID de la boîte avec les ID enregistrés de TC.
3. AA signe un formulaire papier pour le livreur.
4. AA enregistre l'arrivée dans le système.
5. AA retire les sacs de la boîte, les met sur un chariot et les emmène à OE.

Exceptions:

- 2a. L'ID de la boîte ne correspond pas à celui de la société de transport.
- 4a. L'alarme incendie se déclenche et interrompt l'enregistrement.
- 4b. L'ordinateur tombe en panne, on laisse l'argent sur le bureau et on attend que l'ordinateur revienne.

Variations :

- 4'. Avec ou sans « ID- personne ».
- 4''. Avec ou sans « valeur-estimée ».
- 5'. AA laisse les sacs dans la boîte.

AA means "Agent d'Accueil".

OE means " Opérateur d'Enregistrement "

TEST DES CAS D'UTILISATION

- Les tests sont conçus pour exercer les comportements définis (comportement de base, exceptionnel ou alternatif).

COUVERTURE

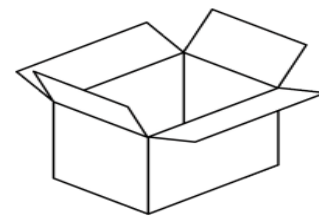
- La couverture est généralement mesurée par le nombre de chemins/ de comportements de cas d'utilisation testés divisé par le nombre total de chemins/ de comportements de cas d'utilisation ; exprimé en pourcentage.
- Par conséquent, **pour couvrir le cas d'utilisation**, nous avons besoin
 - ✓ **Un scénario de test pour couvrir le scénario principal**
 - ✓ **Un scénario de test distinct pour chaque scénario exceptionnel.**
 - ✓ **Un scénario de test distinct pour chaque scénario alternatif.**

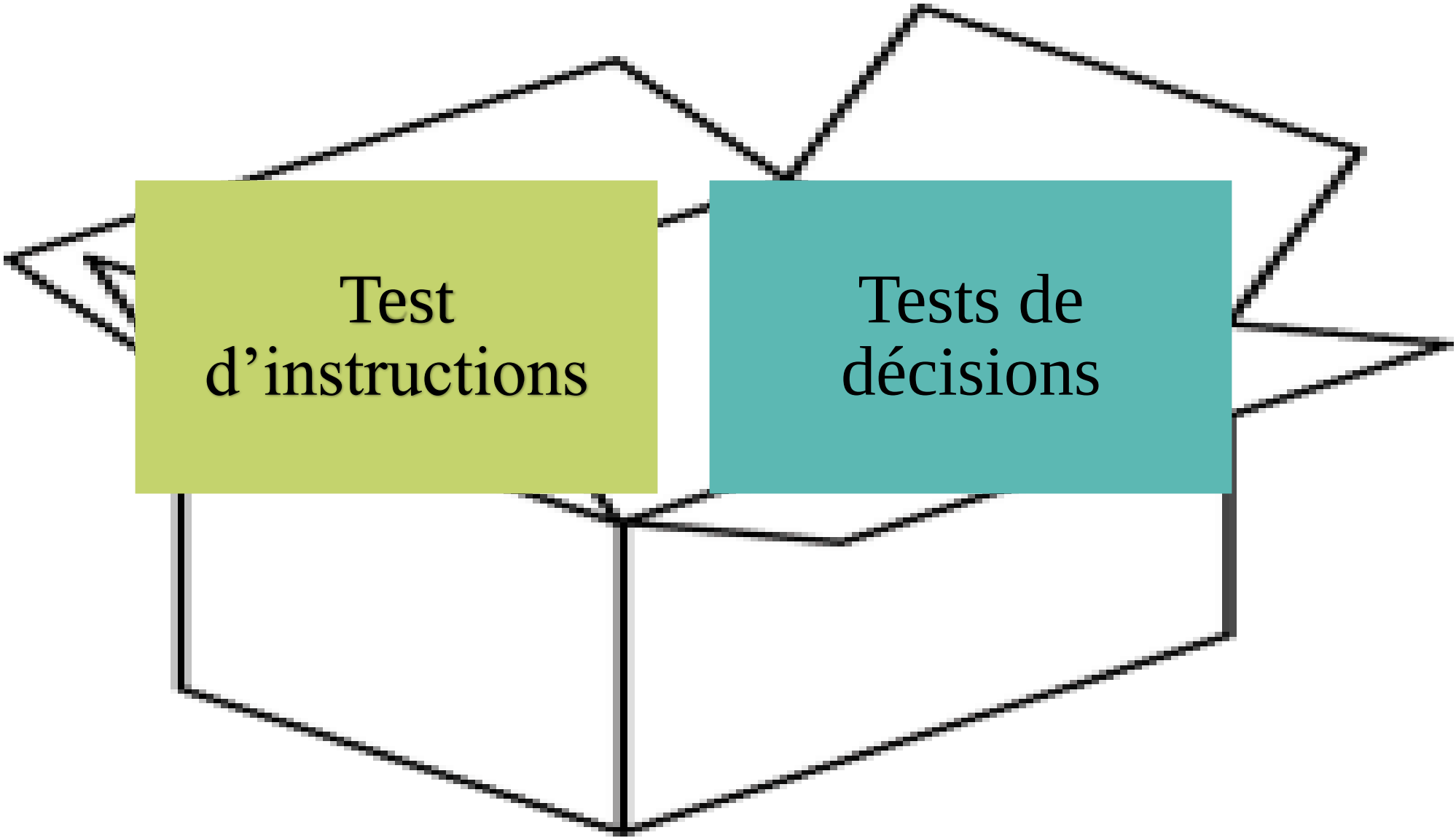
APPLICATION

Use case #001: Register arrival of a box

Numéro du Cas de test	Scénario couverts
1	(scenario principal): 1, 2, 3, 4, 5
2	(scénario exceptionnel 2a): 1, 2a
3	(scénario exceptionnel 4a): 1, 2, 3, 4a
4	(scénario exceptionnel 4b): 1, 2, 3, 4b
5	(scénario alernatif 4'): 1, 2, 3, 4 (sans "personnnne-id"), 5
6	(scénario alernatif 4''): 1, 2, 3, 4 (sans "valeur-estimée"), 5
7	(scénario alernatif 5'): 1, 2, 3, 4, 5 (en laissant les sacs dans la boîte)

TECHNIQUES DE TEST BOÎTE- BLANCHE





Test
d'instructions

Tests de
décisions

FORMATS DES CODES SOURCES

- Le code dans les questions d'examen liées aux techniques de la boîte blanche peut être représenté de deux manières :
 - soit sous la forme d'un pseudocode :
 - Blocs de base : les blocs des instructions exécutées une par une
 - Instructions IF ou IF-ELSE : qui permet de prendre une des deux manières ou plus en fonction de la ou des conditions.
 - Boucle WHILE : qui permet de répéter une séquence d'instructions tant qu'une certaine condition est respectée.
 - soit sous la forme d'un graphe de flux de contrôle

FORMATS DES CODES SOURCES

Exemple de code

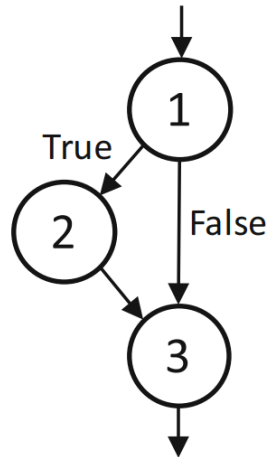
```
1. INPUT x
2. IF (x>0) THEN
3.   PRINT 'The entered number is a positive number'
4.   x := 0
   ELSE
5.   PRINT 'x is negative or 0'
6. END PROGRAM
```

```
1. INPUT x, y
2. IF (x>0) THEN
3.   WHILE (x>0) DO
4.     y := y+1
5.     x := x-1
6.   IF (y>0) THEN
7.     y := 0
   ELSE
8.     y := 1
9.   x := y
10. RETURN x+y
```

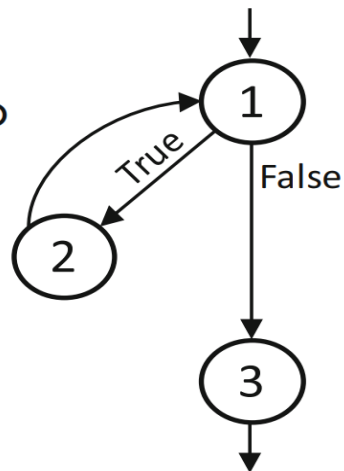

FORMATS DES CODES SOURCES

Graphe de flux de contrôle

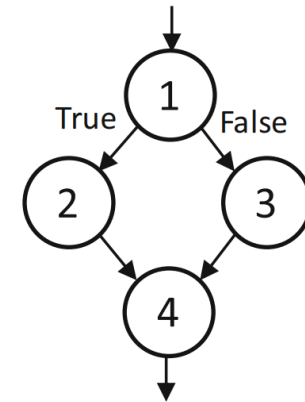
```
1. IF (condition) THEN
2.   Instruction 2
3. END PROGRAM
```



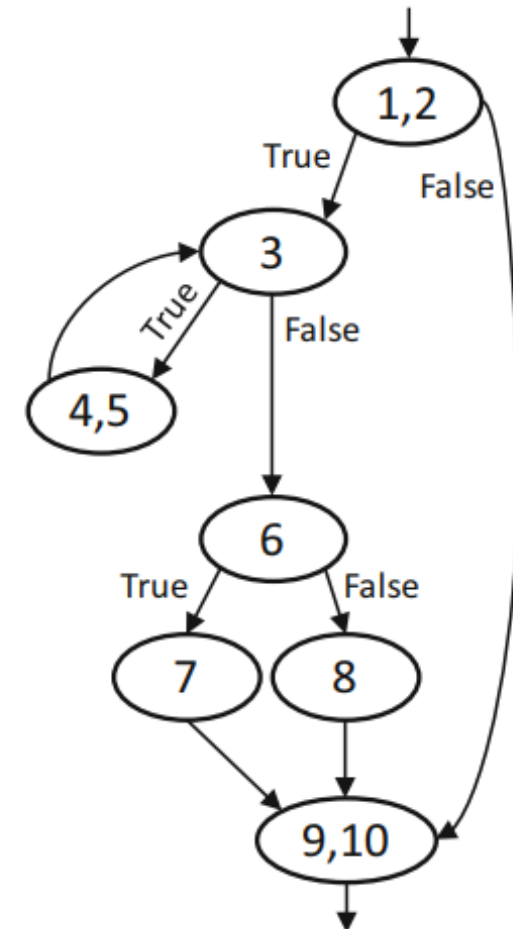
```
1. WHILE (condition) DO
2.   Instruction 2
3. END PROGRAM
```



```
1. IF (condition) THEN
2.   Instruction 2
   ELSE
3.   Instruction 3
4. END PROGRAM
```



```
1. INPUT x, y
2. IF (x>0) THEN
3.   WHILE (x>0) DO
4.     y := y+1
5.     x := x-1
6.   IF (y>0) THEN
7.     y := 0
   ELSE
8.     y := 1
9.   x := y
10. RETURN x+y
```



PRINCIPE

- Les tests boîte-blanche sont basés sur la structure interne de l'objet de test.
- Les techniques de test boîte blanche peuvent être utilisées à tous les niveaux de test, mais **les deux techniques liées au code** dont il est question dans cette partie **sont le plus souvent utilisées au niveau du test des composants**.
 - Exemples: Processus métier au niveau du système, Structures de menus au niveau des tests d'acceptation.
- Il existe des techniques plus avancées qui sont utilisées dans des contextes critiques pour la sûreté, critiques pour la mission ou à fort besoin de fiabilité, et visant à obtenir une couverture plus complète :
 - Exemples : tests de conditions, tests de conditions/décisions, test de conditions multiples

Tests d'instruction

Principe, couverture et exemples

TEST D'INSTRUCTIONS ET COUVERTURE

Principe et mesure de la couverture

- Le test des instructions exerce les instructions exécutables dans le code.
- Il nécessite que chaque instruction exécutable soit testée au moins une fois par au moins un test.
- La couverture est mesurée comme le nombre d'instructions exécutées par les tests, divisé par le nombre total d'instructions exécutables dans l'objet de test, généralement exprimé en pourcentage.

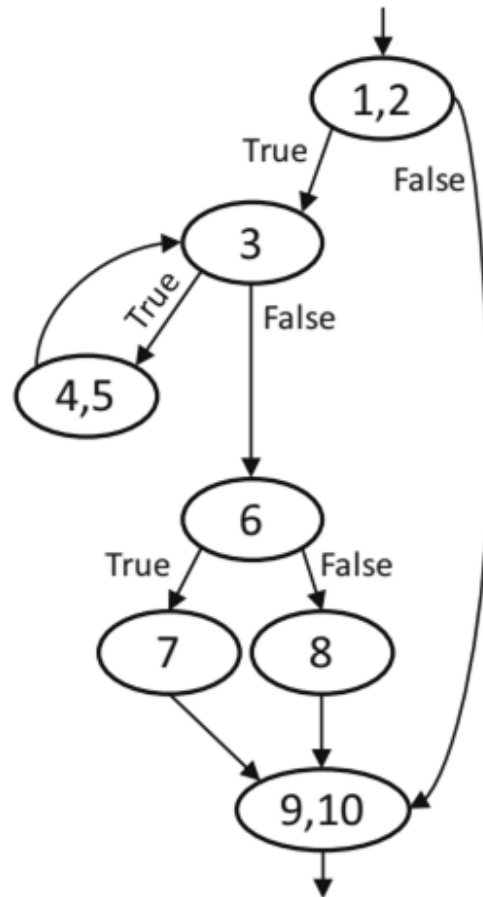
APPORT DES TESTS D'INSTRUCTION

- Lorsque la couverture à 100% des instructions est atteinte, elle garantit que toutes les instructions exécutables du code ont été testées au moins une fois, mais elle ne garantit pas que toutes les décisions ont été testées
- Le test des instructions peut fournir une couverture du code moindre que le test des décisions.
- La couverture des instructions aide à trouver des défauts dans les parties du code qui n'ont pas été exercées par d'autres tests.

EXEMPLE

Test d'instructions

```
1. INPUT x, y
2. IF (x>0) THEN
3.   WHILE (x>0) DO
4.     y := y+1
5.     x := x-1
6.   IF (y>0) THEN
7.     y := 0
8.   ELSE
9.     y := 1
9. x := y
10. RETURN x+y
```



Le nombre minimal de cas de test qui atteignent la couverture de l'instruction (100 %) est de 2, car les lignes 7 et 8 ne peuvent pas être exécutées dans un test.

Ainsi, les deux exemples de cas de test peuvent, par exemple, suivre les chemins suivants :

- test 1 : **1, 2, 3, 4, 5, 3, 6, 7, 9, 10**
- test 2 : **1, 2, 3, 6, 8, 9, 10**

Si notre suite de tests ne contient que le premier test, cette suite atteindra 90 % de couverture des instructions, car le test exerce 9 instructions exécutables sur 10.

Dans le cas du test 2, ce serait 70 %.

Ces deux tests testent ensemble toutes les instructions exécutables, donc ensemble, ils atteignent la couverture de 100% des instructions.

Tests de décisions

Principe, couverture et exemples

TEST DE DÉCISION ET COUVERTURE

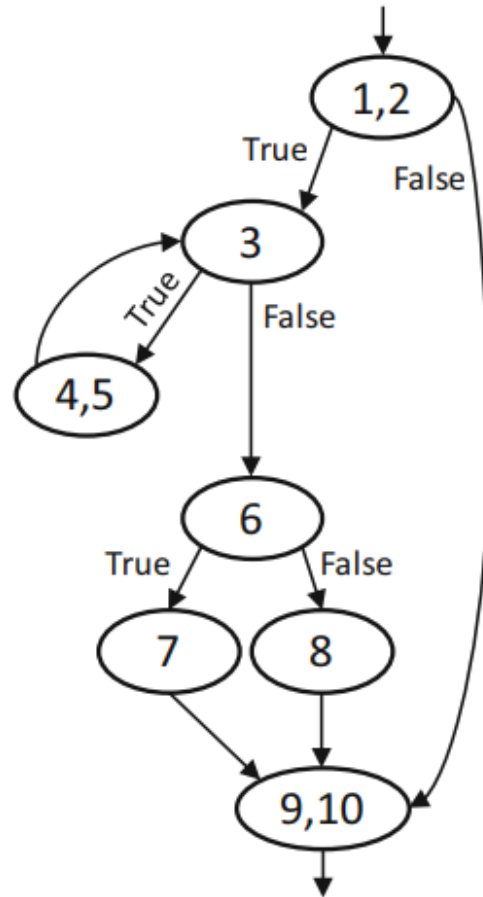
- Le test des décisions exerce les décisions dans le code et teste le code qui est exécuté sur la base des résultats des décisions. Pour cela, les cas de test suivent les flux de contrôle qui se produisent à partir d'un point de décision
 - par exemple, pour une instruction IF, un pour le résultat vrai et un pour le résultat faux ; pour une instruction CASE, des cas de test seraient nécessaires pour tous les résultats possibles, y compris le résultat par défaut).
- La couverture est mesurée comme le nombre de résultats de décision exécutés par les tests, divisé par le nombre total de résultats de décision dans l'objet de test, généralement exprimé en pourcentage

APPORT DES TESTS DE DÉCISIONS

- Lorsque la couverture à 100 % des décisions est atteinte, tous les résultats des décisions sont exécutés, ce qui comprend le test du résultat vrai et aussi du résultat faux, même lorsqu'il n'y a pas d'instruction FALSE explicite.
- La couverture des décisions aide à trouver des défauts dans le code lorsque d'autres tests n'ont pas couvert à la fois les résultats vrais et les résultats faux des décisions.
- **L'obtention d'une couverture à 100 % des décisions garantit une couverture à 100% des instructions (mais pas l'inverse).**

APPLICATION

```
1. INPUT x, y
2. IF (x>0) THEN
3.   WHILE (x>0) DO
4.     y := y+1
5.     x := x-1
6.   IF (y>0) THEN
7.     y := 0
8.   ELSE
9.     y := 1
9.   x := y
10. RETURN x+y
```

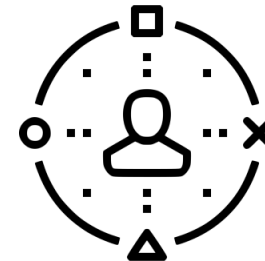


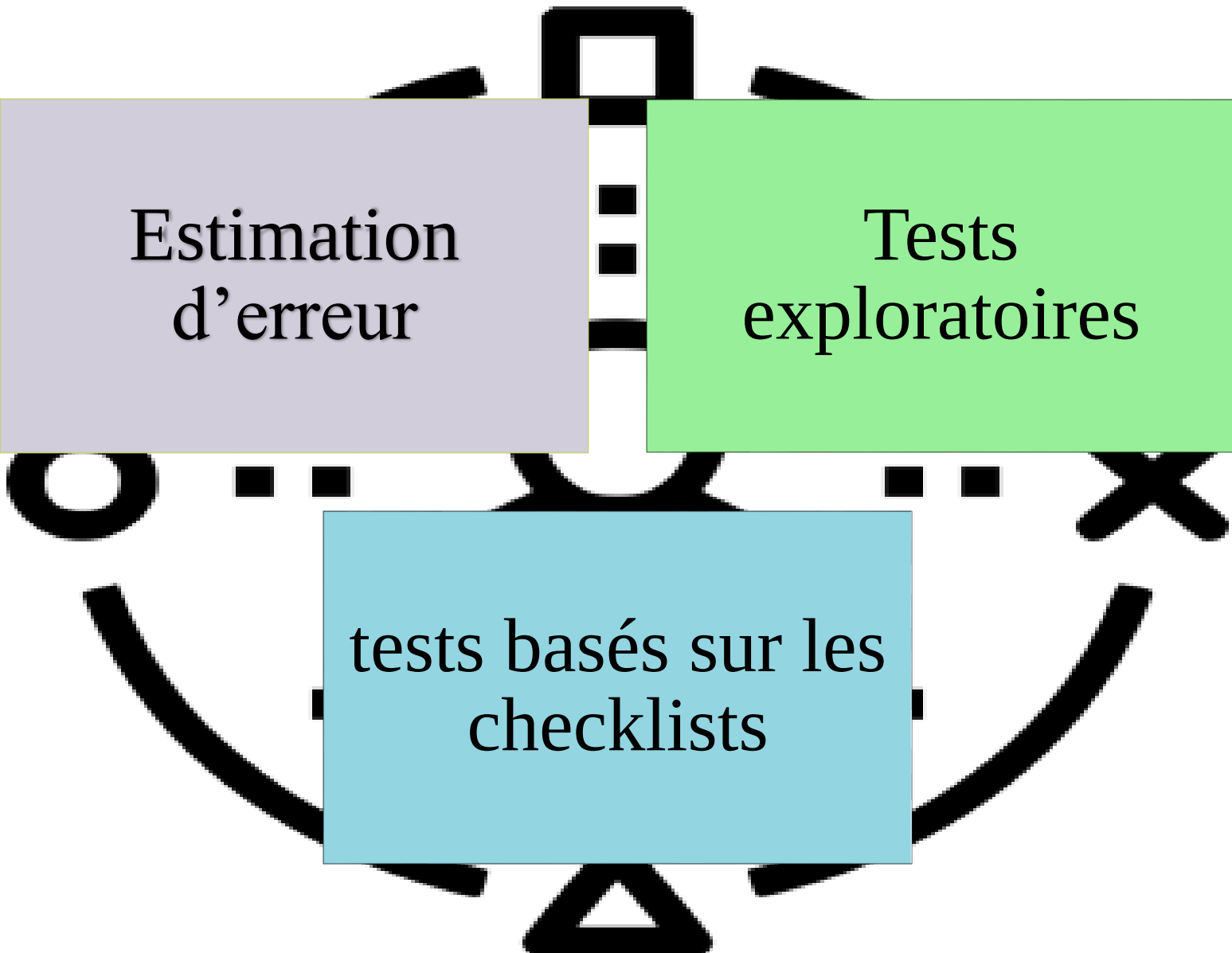
Le nombre minimal de cas de test atteignant la couverture de décision dans cet exemple est de 3. Par exemple,

- Test 1 : **1, 2, 9, 10**
- Test 2 : **1, 2, 3, 4, 5, 3, 6, 7, 9, 10**
- Test 3 : **1, 2, 3, 6, 8, 9, 10**

Le nombre minimal de cas de test dans ce cas est de 3, car deux des conditions de test (résultats de décision) ne peuvent pas être exercées dans un cas de test.

TECHNIQUES DE TEST BASEES SUR L'EXPERIENCE





Estimation
d'erreur

Tests
exploratoires

tests basés sur les
checklists

PRINCIPE

- Lors de l'application de techniques de test basées sur l'expérience, les cas de test sont basés sur les compétences et l'intuition du testeur, ainsi que sur son expérience avec des applications et des technologies similaires.
- Ces techniques peuvent être utiles pour identifier les tests qui n'ont pas été facilement identifiés par d'autres techniques plus systématiques.

COUVERTURE

- Selon l'approche et l'expérience du testeur, ces techniques peuvent atteindre *des degrés de couverture et d'efficacité très variables*.
- **La couverture peut être difficile à évaluer et peut ne pas être mesurable avec ces techniques.**

ESTIMATION D'ERREUR

- L'estimation d'erreur est une technique utilisée pour anticiper les erreurs, les défauts et les défaillances, sur la base des connaissances du testeur, y compris :
 - Comment l'application a fonctionné antérieurement
 - Quels types d'erreurs les développeurs ont tendance à faire
 - Les défaillances qui se sont produites dans d'autres applications
- Une approche méthodique de la technique par estimation d'erreur consiste à créer une liste d'erreurs, de défauts et de défaillances possibles, et à concevoir des tests qui exposeront ces défaillances et les défauts qui les ont causées.
- Ces listes d'erreurs, de défauts et de défaillances peuvent être construites sur la base de l'expérience, de données sur les défauts et les défaillances, ou à partir de connaissances générales sur les causes des défaillances logicielles.

TESTS EXPLORATOIRES

- Dans les tests exploratoires, des tests informels (non prédéfinis) sont conçus, exécutés, enregistrés et évalués dynamiquement pendant l'exécution des tests.
- Les résultats des tests sont utilisés pour en apprendre davantage sur le composant ou le système, et pour créer des tests pour les parties qui pourraient avoir besoin de plus de tests.
- Les tests exploratoires sont parfois réalisés en utilisant des **sessions de test** pour structurer l'activité.
- Lors du test basé sur des sessions, les tests exploratoires sont effectués dans un temps défini, et le testeur utilise une charte de test comprenant les objectifs du test pour guider les tests. Le testeur peut utiliser des fiches de session de test pour documenter les étapes suivies et les constatations réalisées.

TESTS BASÉS SUR LES CHECKLISTS

- Dans les tests basés sur des checklists, les testeurs conçoivent, implémentent et exécutent des tests pour couvrir les conditions de test figurant dans une checklist.
- Au cours de l'analyse, les testeurs créent une nouvelle checklist ou complètent une checklist existante, mais les testeurs peuvent également utiliser une checklist existante sans modification.
- De telles checklists peuvent être construites sur la base de l'expérience, de la connaissance de ce qui est important pour l'utilisateur, ou de la compréhension du pourquoi et du comment des défaillances logicielles.
- Des checklists peuvent être créées pour prendre en charge différents types de tests, y compris les tests fonctionnels et non-fonctionnels.
- En l'absence de cas de tests détaillés, les tests basés sur des checklists peuvent fournir des lignes directrices et un certain degré de cohérence. Comme il s'agit de listes de haut niveau, il est probable qu'il y ait une certaine variabilité dans les tests réels, ce qui pourrait entraîner une plus grande couverture des tests, mais une reproductibilité plus faible.

REFERENCES

